École 2600, module noyau Linux

Rootkit rk2600

Rapport de conception et de mise en œuvre

Novembre 2026

Thomas Abreu Dias · Sofiane Belkessa · Vadim Ablancourt · Cornelius Bircu

1. Objectif et contraintes du projet

Ce rapport présente la conception et la réalisation du rootkit `rk2600` pour le projet offensif de l'École 2600. Le module noyau `rk2600.ko` donne un shell `root` au binaire compagnon `rk2600`, masque des entrées de répertoire et dissimule sa propre présence.

Le sujet suppose un accès initial sur une machine cible dont le noyau Linux charge les modules non signés. Depuis cet accès, `rk2600` s'installe sur le disque, expose ses interfaces de contrôle et se recharge à chaque démarrage.

Le code obéit à quatre contraintes fixées par l'énoncé. Il ne détourne pas l'appel système `kill`, nomme tous ses identifiants en anglais, suit les règles de codage C sécurisé de l'ANSSI et ne reprend aucun code externe.

2. Architecture des trois composants

Le projet réunit trois composants. Le module noyau `rk2600.ko` s'exécute en ring 0 et fournit l'élévation de privilèges, le masquage VFS et le filtrage des appels système. En espace utilisateur, le binaire statique `rk2600` pilote le module. Enfin, l'infrastructure QEMU compile le noyau et construit une image disque Alpine Linux.

Depuis l'espace utilisateur, un opérateur peut piloter `rk2600.ko` avec le binaire `rk2600` ou directement depuis un shell.

2.1 Deux canaux de contrôle entre l'espace utilisateur et le noyau

Le canal principal est le périphérique `misc /dev/.rk_ctl`, qui reçoit des requêtes `ioctl`. Son préfixe `.` le cache d'une commande `ls` sans l'option `-a`, et le hook VFS posé sur `/dev` le retire aussi de `ls -a /dev`. Le second canal est le fichier `/proc/.rk` en écriture seule. Il reçoit des commandes texte depuis un shell standard, sans nécessiter le binaire `rk2600`.

Nous utilisons l'API `miscdevice` avec `MISC_DYNAMIC_MINOR` plutôt qu'un périphérique caractère à numéro majeur fixe. Le noyau alloue lui-même un numéro mineur libre sous le majeur 10, ce qui évite tout conflit avec un pilote existant.

Le tableau suivant résume les cinq composants déployés sur la machine cible.

2.2 Composants déployés sur la cible

Composant	Emplacement	Rôle
Module rk2600.ko	/lib/modules/6.1.188/extra/rk2600.ko	Exécute les hooks noyau et l'élévation de privilèges
Binaire compagnon rk2600	/usr/local/bin/rk2600	Envoie les commandes ioctl et gère l'installation
Périphérique misc	/dev/.rk_ctl	Expose le canal ioctl principal en mode 0666
Entrée procfs	/proc/.rk	Reçoit les commandes texte en écriture seule en mode 0200
Script de persistance	/etc/local.d/snd-compat.start	Recharge rk2600.ko au démarrage via OpenRC

3. Mécanismes du module noyau

3.1 Protocole ioctl partagé

L'en-tête `include/rk2600_protocol.h` définit l'ABI partagée entre `module/rk2600.c` et `companion/companion.c`, avec la version `RK2600_API_VERSION = 2` et l'octet magique `0xA6`. La fonction `rk_ioctl()` traite les six commandes suivantes.

Commande ioctl	Macro	Action dans rk_ioctl()
RK2600_IOC_ELEVATE	_IO	Attribue les privilèges root au processus appelant
RK2600_IOC_STATUS	_IOR	Copie la version d'API et l'état de masquage vers l'espace utilisateur
RK2600_IOC_HIDE	_IOW	Ajoute un nom d'entrée à la liste de masquage
RK2600_IOC_UNHIDE	_IOW	Retire un nom d'entrée de la liste de masquage
RK2600_IOC_MODHIDE	_IO	Retire le module de la liste parcourue par lsmod
RK2600_IOC_MODSHOW	_IO	Réinsère le module dans la liste parcourue par lsmod

3.2 Commandes texte sur /proc/.rk

Au chargement, `rk_init()` crée `/proc/.rk` en mode `0200` avec `proc_create()`. La fonction `proc_rk_write()` traite cinq commandes texte :

- `+nom` ajoute `nom` à la liste des entrées masquées via `hide_name()`.
- `-nom` retire `nom` de la liste des entrées masquées via `show_name()`.
- `hide` retire le module de `lsmod`, et `unhide` l'y réinsère.
- `flush` vide toute la liste des entrées masquées via `flush_hidden()`.

Le hook `rk_proc_iterate` filtre `/proc` avec `rk_filldir`, et `rk_init()` ajoute `.rk` à la liste des noms masqués dès le démarrage. La commande `ls -a /proc` n'affiche donc pas `/proc/.rk`.

3.3 Masquage des entrées de répertoire dans le VFS

Pour masquer un fichier ou un répertoire, `install_dir_hook()` ouvre `/`, `/proc` et `/dev`, puis remplace le pointeur `iterate_shared` de leur structure `file_operations` par `rk_root_iterate`, `rk_proc_iterate` et `rk_dev_iterate`. Chacune de ces fonctions enveloppe le `dir_context` de l'appelant et substitue `rk_filldir` au callback `actor` d'origine.

Les structures `file_operations` du noyau résident dans des pages mémoire en lecture seule. Pour réécrire `iterate_shared` sans provoquer de faute de page en ring 0, `wp_disable()` coupe la préemption sur le cœur courant et efface le bit `X86_CR0_WP` du registre `CR0` :

```
cr0 = wp_disable();/* preempt_disable() + CR0 & ~X86_CR0_WP */
h->fops->iterate_shared = h->hooked_iterate;
wp_restore(cr0);/* restaure CR0 + preempt_enable() */
```

Le module stocke les noms cachés dans la liste chaînée `hidden_names`, protégée par le spinlock `hidden_lock`. À chaque entrée lue par le système de fichiers, `rk_filldir()` appelle `name_is_hidden()`. Si le nom figure dans `hidden_names`, `rk_filldir()` retourne `true` sans appeler le callback d'origine. Les outils fondés sur `getdents64`, comme `ls` et `find`, ne reçoivent jamais l'entrée masquée.

Les hooks posés sur `/proc` et `/dev` masquent `.rk` et `.rk_ctl`. Comme tous les répertoires d'un même système de fichiers `ext4` partagent la même table `ext4_dir_operations`, le hook posé sur `/` filtre tous les répertoires `ext4` de la machine. En revanche, le module n'inclut pas de commande dédiée au masquage d'un PID numérique dans `/proc`, comme l'explique la section 9.2.

3.4 Retrait du module de la liste de lsmod

La commande `lsmod` lit `/proc/modules`, qui parcourt la liste doublement chaînée globale des modules chargés. Pour soustraire `rk2600` à ce parcours, `do_hide_module()` sauvegarde le pointeur `THIS_MODULE->list.prev` dans `saved_prev`, puis détache le module avec `list_del_init(&THIS_MODULE->list)`. La commande `RK2600_IOC_MODSHOW` appelle `do_show_module()`, qui réinsère le module après `saved_prev` avec `list_add()`.

La fonction `rk_init()` appelle `do_hide_module()` dès le chargement. Le module n'appelle pas `kobject_del()` afin de permettre un déchargement propre par `rmmod` après réinsertion. Pour cacher aussi le répertoire `/sys/module/rk2600` aux parcours de répertoires tant que `/sys` n'est pas inspecté directement, `rk_init()` ajoute `rk2600` à `hidden_names`.

3.5 Interception ftrace de read, write et sendfile64

Même si `snd-compat.start` est invisible dans `ls /etc/local.d/`, un administrateur qui connaît ce chemin pourrait lire ou écraser le fichier. Le module détourne donc `__x64_sys_read`, `__x64_sys_write` et `__x64_sys_sendfile64` avec `ftrace`.

Résolution des symboles et pose des hooks ftrace.

Depuis Linux 5.7, le noyau n'exporte plus `kallsyms_lookup_name`. La fonction `install_syscall_hooks()` pose une `kprobe` temporaire sur `kallsyms_lookup_name`, lit son adresse dans `kp.addr`, puis retire aussitôt la `kprobe`. Pour chaque entrée du tableau `hooks[]`, `install_one_hook()` résout l'adresse de la fonction `__x64_sys_*`, cible cette adresse avec `ftrace_set_filter_ip()` et enregistre le callback `fh_callback()` avec les drapeaux `FTRACE_OPS_FL_SAVE_REGS` et `FTRACE_OPS_FL_IPMODIFY`. À chaque entrée dans le syscall, `fh_callback()` redirige `regs->ip` vers notre fonction hook, sans toucher à `sys_call_table`.

```
struct kprobe kp = { .symbol_name = "kallsyms_lookup_name" };
register_kprobe(&kp);
klnfn = (kallsyms_lookup_name_fn)kp.addr;
unregister_kprobe(&kp);
```

Identification du script par inode et superbloc.

Comparer des chemins à chaque appel système exposerait le filtre aux liens symboliques et aux renommages. La fonction `resolve_persist_file()` résout `/etc/local.d/snd-compat.start` avec `kern_path()` et enregistre son numéro d'inode `persist_ino` ainsi que l'identifiant de périphérique `persist_dev`. À chaque appel intercepté, `fd_is_persist()` récupère la structure `struct file` du descripteur via `fget(fd)` et compare `f_inode->i_ino` et `f_inode->i_sb->s_dev`. Si le script n'existe pas encore au chargement du module, `fd_is_persist()` relance `resolve_persist_file()` aux appels suivants jusqu'à sa création.

Comportement des trois hooks.

Après l'appel à `orig_sys_read()`, `rk_sys_read()` inspecte le tampon utilisateur lorsque le descripteur cible `snd-compat.start`. La fonction `filter_persist_buf()` copie les octets lus en mémoire noyau et retire toute ligne contenant `insmod` ou `rk2600` avant de recopier le tampon filtré vers l'espace utilisateur. Sur ce même fichier, `rk_sys_write()` retourne le nombre d'octets `regs->dx` demandé par l'appelant sans appeler `orig_sys_write()`, ce qui ignore toute écriture. Enfin, comme `cat` sous BusyBox utilise `sendfile64()` avant `read()`, `rk_sys_sendfile64()` retourne `-EINVAL` lorsque `snd-compat.start` est le descripteur source `regs->si`. Ce code d'erreur force `cat` à relire le fichier avec `read()`, où `rk_sys_read()` filtre la ligne `insmod` :

```
# Contenu sur le disque dans /etc/local.d/snd-compat.start :
#!/bin/sh
# Sound compatibility layer
printk_lvl=$(cat /proc/sys/kernel/printk | cut -f1)
echo 1 > /proc/sys/kernel/printk
```

```
/sbin/insmod /lib/modules/6.1.188/extra/rk2600.ko 2>/dev/null
echo "$printk_lvl" > /proc/sys/kernel/printk
dmesg -c > /dev/null 2>&1

# Contenu renvoyé à cat après filtrage par rk_sys_read() :
#!/bin/sh
# Sound compatibility layer
printk_lvl=$(cat /proc/sys/kernel/printk | cut -f1)
echo 1 > /proc/sys/kernel/printk
echo "$printk_lvl" > /proc/sys/kernel/printk
dmesg -c > /dev/null 2>&1
```

Justification de ftrace.

L'énoncé interdit de détourner `kill`. Pour intercepter `read`, `write` et `sendfile64`, `ftrace` offre deux avantages sur la réécriture de `sys_call_table` : il s'appuie sur l'infrastructure officielle de traçage du noyau et ne modifie aucune page en lecture seule via `CR0.WP`.

Angles morts connus.

Le module ne filtre pas `pread64`, `readv`, `preadv2`, `splice` ni `mmap`. Par ailleurs, ouvrir le script avec `O_TRUNC` vide le fichier dans le VFS avant tout appel à `write()`. Les sections 9.3 et 9.4 détaillent ces deux limites.

4. Élévation de privilèges par commit_creds

Sur réception de `RK2600_IOC_ELEVATE`, la fonction `do_elevate()` appelle `prepare_kernel_cred(NULL)`. Sur Linux 6.1.188, un argument `NULL` duplique les privilèges `init_cred` du noyau, soit `uid = 0`, `gid = 0` et toutes les capacités POSIX, puis `commit_creds()` les applique au processus appelant :

```
new = prepare_kernel_cred(NULL);
if (!new)
return -ENOMEM;
return commit_creds(new);
```

Dans le binaire compagnon, `cmd_privesc()` envoie `RK2600_IOC_ELEVATE` sur `/dev/.rk_ctl`, vérifie que `geteuid() == 0`, puis exécute `/bin/sh` avec `execlp()`. Le module enregistre `/dev/.rk_ctl` avec les permissions `0666` pour qu'un compte non privilégié puisse ouvrir le périphérique et demander l'élévation.

Ce mode `0666` permet à tout utilisateur local qui connaît le chemin `/dev/.rk_ctl` d'obtenir un shell `root`. Ce réglage répond au scénario de démonstration depuis le compte `user`. La section 9.1 détaille comment restreindre cet accès par un échange de jeton HMAC.

Appeler `prepare_kernel_cred()` et `commit_creds()` évite d'écraser directement les champs `uid` et `gid` dans `task_struct->cred`. Le noyau maintient ainsi lui-même les compteurs de références RCU et les structures de sécurité LSM associées aux identifiants du processus.

5. Persistance au démarrage avec OpenRC

Sur Alpine Linux, le service OpenRC `local` exécute à chaque démarrage les scripts `.start` exécutables présents dans `/etc/local.d/`. La commande `rk2600 install` configure cette persistance en quatre étapes :

- Copie `rk2600.ko` dans `/lib/modules/<version>/extra/rk2600.ko` via `copy_file()`.
- Écrit `/etc/local.d/snd-compat.start` en mode `0755`, sous un nom qui imite un script de compatibilité audio.
- Active le service `local` dans le runlevel `default` avec `rc-update add local default`.
- Charge immédiatement le module avec `insmod`.

Au démarrage, `/etc/local.d/snd-compat.start` abaisse le seuil des messages console du noyau à 1 dans `/proc/sys/kernel/printk` avant d'appeler `/sbin/insmod`, restaure ensuite la valeur initiale, puis vide le tampon `dmesg` :

```
printk_lvl=$(cat /proc/sys/kernel/printk | cut -f1)
echo 1 > /proc/sys/kernel/printk
/sbin/insmod /lib/modules/6.1.188/extra/rk2600.ko 2>/dev/null
echo "$printk_lvl" > /proc/sys/kernel/printk
dmesg -c > /dev/null 2>&1
```

Dès son initialisation, `rk_init()` appelle `hide_name()` sur cinq entrées : `snd-compat.start`, `rk2600.ko`, `rk2600`, `.rk` et `.rk_ctl`. Les commandes `ls /etc/local.d/`, `ls /lib/modules/6.1.188/extra/`, `ls /sys/module/`, `ls -a /proc/` et `ls -a /dev/` ne montrent donc aucun artefact du rootkit.

6. Binaire compagnon en espace utilisateur

Le fichier `companion/companion.c` compile en un binaire statique `companion/rk2600`, installé sous `/usr/local/bin/rk2600` dans l'image disque. Il dépend uniquement de la bibliothèque C standard, ce qui permet de le lier statiquement avec `musl` ou `glibc` via le type `ioctl_request_t`. Le binaire expose les sept sous-commandes suivantes.

Sous-commande	Arguments	Action
status	aucun	Affiche la version d'API et indique si le module est masqué
privesc	aucun	Envoie RK2600_IOC_ELEVATE, vérifie geteuid() == 0 et lance /bin/sh
hide	<nom>	Masque l'entrée <nom> dans les répertoires couverts par le hook VFS
unhide	<nom>	Retire <nom> de la liste des entrées masquées
install	[ko]	Copie le .ko, crée snd-compat.start, active OpenRC local et lance insmod
uninstall	aucun	Révèle le module, exécute rmmod et supprime le .ko ainsi que snd-compat.start
test	aucun	Exécute la suite embarquée de 12 tests fonctionnels

Avant d'appeler `ioctl()`, `name_is_valid()` vérifie l'argument de `hide` et `unhide` : le nom doit mesurer entre 1 et 255 octets, soit `RK2600_NAME_MAX`, ne contenir aucun caractère `/` et différer de `.` et `..`. Lors de `rk2600 install`, la fonction interne `copy_file()` copie `rk2600.ko` avec une boucle d'écriture complète sur `read()` et `write()`, sans invoquer `cp`.

7. Environnement de test et image virtuelle

L'environnement de test exécute Linux 6.1.188 et Alpine Linux sous `qemu-system-x86_64`. Le `Makefile` racine délègue la construction et les tests aux six scripts du dossier `infra/scripts/` décrits ci-dessous.

Script	Rôle
common.sh	Définit les chemins, les versions, les variables .env et les fonctions partagées
build_kernel.sh	Télécharge, configure et compile Linux 6.1.188 pour les profils base et debug
build_rootfs.sh	Construit le rootfs BusyBox pour les modes de développement initramfs et NFS
build_disk.sh	Crée disk.img avec ext4, Alpine, OpenRC, GRUB BIOS et préinstalle rk2600
run.sh	Démarre la machine virtuelle QEMU en mode initramfs, NFS ou disque et attache GDB
test_vm.exp	Démarre rk2600.qcow2 sous Expect et vérifie les 12 tests de rk2600 test

Le script `build_disk.sh` génère une image brute `infra/disk.img` de 1 Gio, puis la cible `make qcow2` la convertit en une image QCOW2 compressée `rk2600.qcow2` :

```
make disk && make qcow2
qemu-img check rk2600.qcow2
make test-vm
```

Le fichier livré `rk2600.qcow2` pèse 24 Mio sur l'hôte, valeur mesurée avec `du -h rk2600.qcow2`, pour une capacité virtuelle de 1 Gio. La commande `qemu-img check rk2600.qcow2` confirme l'absence d'erreur dans l'image, et `make test-vm` valide les 12 vérifications de `rk2600 test` sous QEMU.

L'image Alpine contient deux comptes configurés à partir du fichier `.env` :

- `root` sert à l'administration et au chargement manuel du module avec `insmod`.
- `user` sert à tester l'élévation de privilèges et le masquage depuis un compte non privilégié.

Le guide `docs/USER.md` documente les mots de passe et les commandes de vérification.

8. Scénarios d'accès initial sur systèmes non signés

Le sujet suppose un accès initial `root` sur une machine dont le noyau accepte les modules non signés. Cette section présente cinq cas réels qui réunissent ces deux conditions dans les parcs Linux en production, classés du plus fréquent au plus ciblé.

8.1 Objets connectés et équipements réseau grand public

Les routeurs sous OpenWrt, les NAS Synology ou QNAP, les box opérateur et les caméras IP tournent le plus souvent sur un noyau Linux compilé par le fabricant avec

`CONFIG_MODULE_SIG=n`. Le constructeur évite ainsi de gérer une autorité de certification interne et vérifie uniquement la signature globale de l'image firmware lors des mises à jour.

Dans ce contexte, un attaquant exploite généralement une vulnérabilité sur l'interface web d'administration ou sur un démon UPnP exposé pour obtenir un shell root et charger directement son module `.ko`*.*

8.2 Systèmes industriels et équipements médicaux

Les automates programmables sous Linux, les superviseurs SCADA et les appareils d'imagerie médicale conservent pendant des années des branches noyau 3.x ou 4.x figées par leur certification réglementaire. Les intégrateurs désactivent presque toujours Secure Boot sur ces machines pour autoriser les interventions de maintenance sur site.

Un attaquant obtient alors son accès initial par une clé USB de maintenance branchée sur site ou par un tunnel VPN de télémaintenance mal cloisonné.

8.3 Postes de développement et agents d'intégration continue auto-hébergés

De nombreux développeurs désactivent Secure Boot sur leur station de travail pour démarrer en double amorçage, charger des pilotes propriétaires non signés ou tester leurs propres noyaux. De même, les agents d'intégration continue auto-hébergés tournent souvent dans des machines virtuelles Linux sans Secure Boot, où les tâches de build disposent de `sudo` sans mot de passe.

L'accès initial provient ici d'une dépendance compromise, installée via `npm install`*,* `pip install` *ou* `cargo install`*, ou d'une pull request malveillante qui exécute du code arbitraire sur l'agent CI.*

8.4 Machines avec Secure Boot actif mais clé MOK exposée

Lorsqu'un administrateur garde Secure Boot actif mais installe des modules tiers compilés par DKMS comme `nvidia-dkms`, `zfs-dkms` ou `VirtualBox`, il enrôle une clé MOK dans `mokutil`. Or la clé privée correspondante reste souvent stockée en clair sur le disque, dans `/var/lib/dkms/mok.key` ou `/root/.mok/`. Quiconque lit cette clé peut signer `rk2600.ko` avec `sign-file` et franchir le contrôle de signature du noyau.

L'attaquant entre d'abord sous un compte non privilégié, obtient root via une vulnérabilité locale du noyau ou d'un binaire SUID comme `pkexec`*, puis signe son module avec la clé MOK locale.*

8.5 Centres de données et noyaux maintenus en interne

Certains hébergeurs et opérateurs cloud compilent leur propre branche noyau pour leurs hyperviseurs et leurs machines virtuelles internes. Une partie de ces flottes désactive Secure Boot et `CONFIG_MODULE_SIG_FORCE` afin de charger des modules de diagnostic ou de réseau sans maintenir une chaîne de signature par environnement.

L'accès initial résulte alors de l'hameçonnage d'un compte d'administration ou de la compromission d'un serveur de déploiement interne.

8.6 Contre-mesures face à ce modèle d'attaque

Ces cinq cas montrent que le chargement d'un module non signé après compromission initiale reste réaliste hors des serveurs durcis avec Secure Boot et `CONFIG_MODULE_SIG_FORCE=y`. Pour bloquer ou détecter un tel rootkit, un défenseur doit combiner quatre mécanismes : le mode `integrity` ou `confidentiality` de `kernel lockdown`, qui interdit les modules non signés et les `kprobes`, la surveillance `auditd` sur les appels système `init_module` et `finit_module`, le contrôle d'intégrité `IMA` et `EVM` sur `/etc/local.d/` et `/lib/modules/`, et la vérification périodique des pointeurs `file_operations` et des filtres `ftrace`.

9. Limites connues

Le module `rk2600.ko` remplit toutes les exigences fonctionnelles sur l'environnement de référence, soit Alpine Linux sur Linux 6.1.188 `x86_64`, mais comporte huit limites techniques ou de conception.

9.1 Accès non authentifié sur /dev/.rk_ctl

Le périphérique `/dev/.rk_ctl` est créé avec le mode `0666` afin que le compte `user` puisse exécuter `rk2600 privesc` et `rk2600 test` sans configuration préalable. En contrepartie, tout processus local qui découvre ce chemin peut appeler `RK2600_IOC_ELEVATE`. Pour restreindre cet accès, `RK2600_IOC_STATUS` pourrait émettre un nonce aléatoire à usage unique, et `RK2600_IOC_ELEVATE` exigerait un code HMAC calculé par `rk2600` avec une clé prépartagée.

9.2 Couverture partielle des systèmes de fichiers montés

La fonction `rk_init()` remplace `iterate_shared` sur les structures `file_operations` associées à `/`, `/proc` et `/dev`. Comme le noyau partage une même structure `file_operations` entre tous les répertoires d'un même type de système de fichiers, le hook posé sur `/` couvre tous les répertoires `ext4`, mais ne filtre ni `tmpfs` sous `/run`, ni `sysfs` sous `/sys`, ni `nfs`. Intercepter directement `__x64_sys_getdents64` et `__x64_sys_getdents` via `ftrace` filtrerait tous les systèmes de fichiers d'un seul coup.

9.3 Appels de lecture non filtrés sur le script de persistance

Le module filtre `__x64_sys_read` et bloque `__x64_sys_sendfile64`, mais n'intercepte pas `pread64`, `readv`, `preadv2`, `splice` ni `mmap`. Un programme qui projette `/etc/local.d/snd-compat.start` en mémoire avec `mmap()`, ou qui le lit avec `pread64()`, voit donc la ligne `/sbin/insmod` non filtrée.

9.4 Troncature à l'ouverture et renommage du script de persistance

Le hook `rk_sys_write()` ignore les octets écrits dans `snd-compat.start` et retourne un succès apparent. Toutefois, si un processus ouvre le fichier avec le drapeau `O_TRUNC`, par exemple lors d'une redirection shell `> /etc/local.d/snd-compat.start`, le VFS vide le fichier dès l'appel `openat()`, avant tout appel à `write()`. De même, `unlink()` ou `rename()` sur `/etc/local.d/snd-compat.start` ne sont pas interceptés.

9.5 Exposition des symboles dans /proc/kallsyms et /proc/kcore

Même après `do_hide_module()`, les symboles du module comme `rk_sys_read`, `rk_root_iterate`, `rk_ioctl` et `rk_fops` restent visibles dans `/proc/kallsyms`, dans `/proc/kcore` et dans `/sys/kernel/tracing/enabled_functions`. Une commande `grep rk_ /proc/kallsyms` révèle donc la présence du module. Étendre `rk_sys_read()` pour filtrer les lignes de `/proc/kallsyms` et de `enabled_functions` supprimerait cet indice.

9.6 Écriture directe dans file_operations via CR0.WP

Les hooks `read`, `write` et `sendfile64` passent par `ftrace`, mais les hooks de répertoires modifient encore les pointeurs `iterate_shared` en désactivant temporairement `X86_CR0_WP`. Sur un hyperviseur ou un noyau doté d'une protection d'intégrité du code noyau, l'écriture dans une structure `const struct file_operations` ou l'instruction `mov %cr0` peut déclencher une alerte. Porter le filtrage des répertoires sur `ftrace` au niveau de `__x64_sys_getdents64` éliminerait toute manipulation de `CR0`.

9.7 Dépendance à l'architecture x86_64

Le code cible uniquement l'architecture `x86_64` : il utilise les noms de symboles `__x64_sys_*`, lit les arguments d'appels système dans `regs->di`, `regs->si` et `regs->dx`, et modifie le bit `X86_CR0_WP` du registre `CR0`. Un portage vers `ARM64` demanderait d'adapter les registres `pt_regs`, les noms `__arm64_sys_*` et le mécanisme de contournement des pages en lecture seule.

9.8 Détection par contrôle d'intégrité des pointeurs file_operations

Un agent de sécurité noyau qui enregistre au démarrage l'adresse des pointeurs `f_op->iterate_shared` de `/`, `/proc` et `/dev` détecte immédiatement leur remplacement par des adresses situées hors du texte noyau principal. Là encore, un hook `ftrace` sur `getdents64` laisserait les tables `file_operations` intactes.

10. Compilation et compatibilité du noyau

10.1 Noyau cible Linux 6.1.188

Le script `infra/scripts/build_kernel.sh` compile Linux 6.1.188, branche stable à support long. Le module `rk2600.ko` doit être compilé contre l'arbre `infra/build` exact du noyau installé dans l'image disque, faute de quoi `insmod` échoue avec l'erreur `Invalid module format`.

La configuration générée par `build_kernel.sh` active les options suivantes :

- `CONFIG_MODULES=y` et `CONFIG_MODULE_UNLOAD=y` autorisent le chargement et le déchargement de modules sans imposer `CONFIG_MODULE_SIG_FORCE`.
- `CONFIG_KPROBES=y`, `CONFIG_FTRACE=y`, `CONFIG_DYNAMIC_FTRACE=y` et `CONFIG_DYNAMIC_FTRACE_WITH_REGS=y` fournissent l'infrastructure nécessaire à `register_kprobe()` et `register_ftrace_function()`.

- `CONFIG_SECURITY_LOCKDOWN_LSM=y` est compilé dans le noyau mais reste inactif par défaut au démarrage, tandis que le profil `debug` ajoute `nokaslr` sur la ligne de commande GRUB pour faciliter le débogage GDB.

10.2 Commandes de compilation et de vérification

Exécutez les commandes suivantes depuis la racine du dépôt pour tout compiler et vérifier l'image :

```
make kernel PROFILE=base# compile Linux 6.1.188 dans infra/build
make module# compile module/rk2600.ko contre infra/build
make companion# compile le binaire statique companion/rk2600
make disk# construit infra/disk.img avec Alpine, OpenRC et GRUB
make qcow2# compresse infra/disk.img en rk2600.qcow2
make test-companion# exécute les tests CLI hors VM
make test-vm# démarre rk2600.qcow2 et valide les 12 tests
```

10.3 Licence GPL et résolution des symboles

Le fichier `module/rk2600.c` déclare `MODULE_LICENSE("GPL")`. Sans cette déclaration, le noyau refuse de lier le module aux fonctions internes réservées aux modules GPL, comme `register_kprobe()`, `ftrace_set_filter_ip()` et `register_ftrace_function()`.

11. Répartition des tâches

Chaque membre du groupe a pris en charge un sous-système principal et a co-révisé les interfaces partagées. Le tableau suivant détaille la répartition des travaux.

Membre	Domaine principal	Tâches réalisées

Thomas Abreu Dias	Module noyau, masquage VFS	Hooks iterate_shared sur /, /proc et /dev Liste chaînée hidden_names et spinlock hidden_lock Contournement de X86_CR0_WP via wp_disable et wp_restore Callback unifié rk_filldir
Sofiane Belkessa	Module noyau, ioctl et élévation	Périphérique misc /dev/.rk_ctl et gestionnaire rk_ioctl() Protocole partagé include/rk2600_protocol.h Élévation de privilèges do_elevate() avec prepare_kernel_cred et commit_creds Masquage et réinsertion du module dans lsmod
Vadim Ablancourt	Binaire compagnon, hooks ftrace et tests	Binaire compagnon companion/companion.c et ses 7 sous-commandes Interface /proc/.rk et gestionnaire proc_rk_write() Hooks ftrace read, write et sendfile64 et kprobe kallsyms_lookup_name Tests unitaires CLI, suite rk2600 test et guide docs/USER.md
Cornelius Bircu	Infrastructure QEMU et système de build	Scripts infra/scripts/ pour le noyau, le rootfs, le disque et QEMU Image disque Alpine Linux, OpenRC et GRUB BIOS dans rk2600.qcow2 Script de persistance OpenRC /etc/local.d/snd-compat.start Automatisation des tests QEMU sous Expect avec test_vm.exp

12. Ratio de contribution

Un ratio de 100 % indique une contribution conforme aux attentes du groupe. La note collective est multipliée par ce ratio pour calculer la note individuelle de chaque étudiant.

Membre	Ratio (%)
Thomas Abreu Dias	100 %
Sofiane Belkessa	100 %
Vadim Ablancourt	100 %

Cornelius Bircu	100 %

Les quatre membres du groupe ont validé cette répartition à parts égales.